MACHINE LEARNING AND DEEP LEARNING

Lab 05: Transformers

Dr. Giacomo Lucato, a.y. 2025-2026

TRANSFORMERS

- **Transformers** were originally introduced to solve **sequence modeling.**
- Previous models, like RNNs and LSTMs, present two major problems: they are **slow**, because they have to process one token at a time, and they tend to **forget past information** as the input sequence grows longer.
- Transformers address these issues introducing two novelties:
  - **Parallelism:** Transformers process the input sequence all at once. This makes them incredibly fast, especially on modern hardware.
  - **Self-Attention:** it allows the model to weigh the importance of different words in a sentence relative to a specific word.

TRANSFORMERS ARCHITECTURE

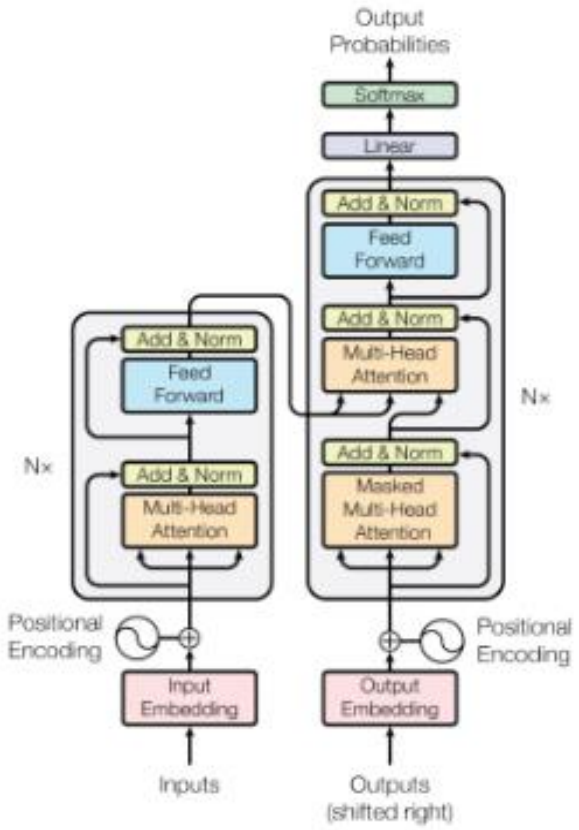
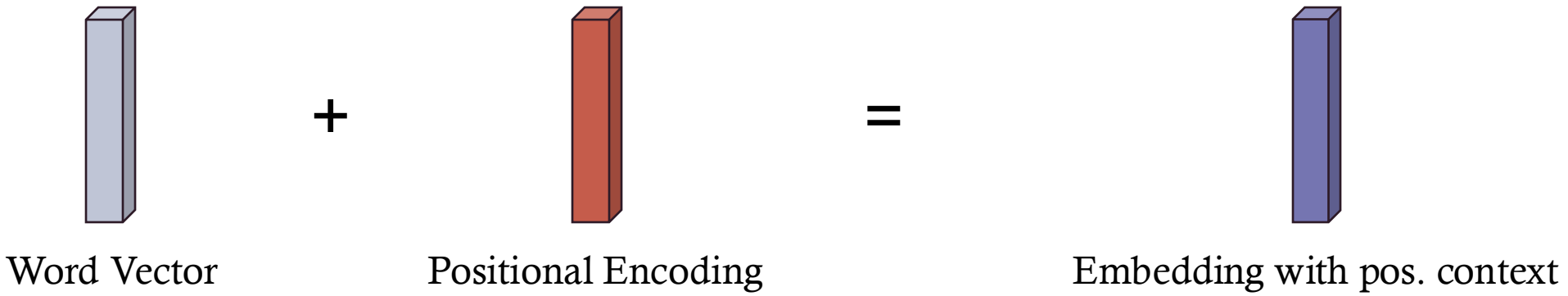


Figure 1: The Transformer - model architecture.

POSITIONAL ENCODING

- Because Transformers process input data in parallel, they become **permutation invariant:** they lose the sense of order among words.
- To overcome this problem, input embeddings are combined with **positional embeddings**, obtained by passing the input sequence through a **positional encoder**, that creates a **unique signature** for each position through a combination of **sine and cosine functions.**



MULTI-HEAD SELF ATTENTION

- This block is the **core** of the Transformer architecture: it uses a special attention mechanism, known as **self-attention,** to relate each word in the input with other words, allowing the model to capture the **context of other relevant words** for the one which is currently processed.
- This is done through the following operation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

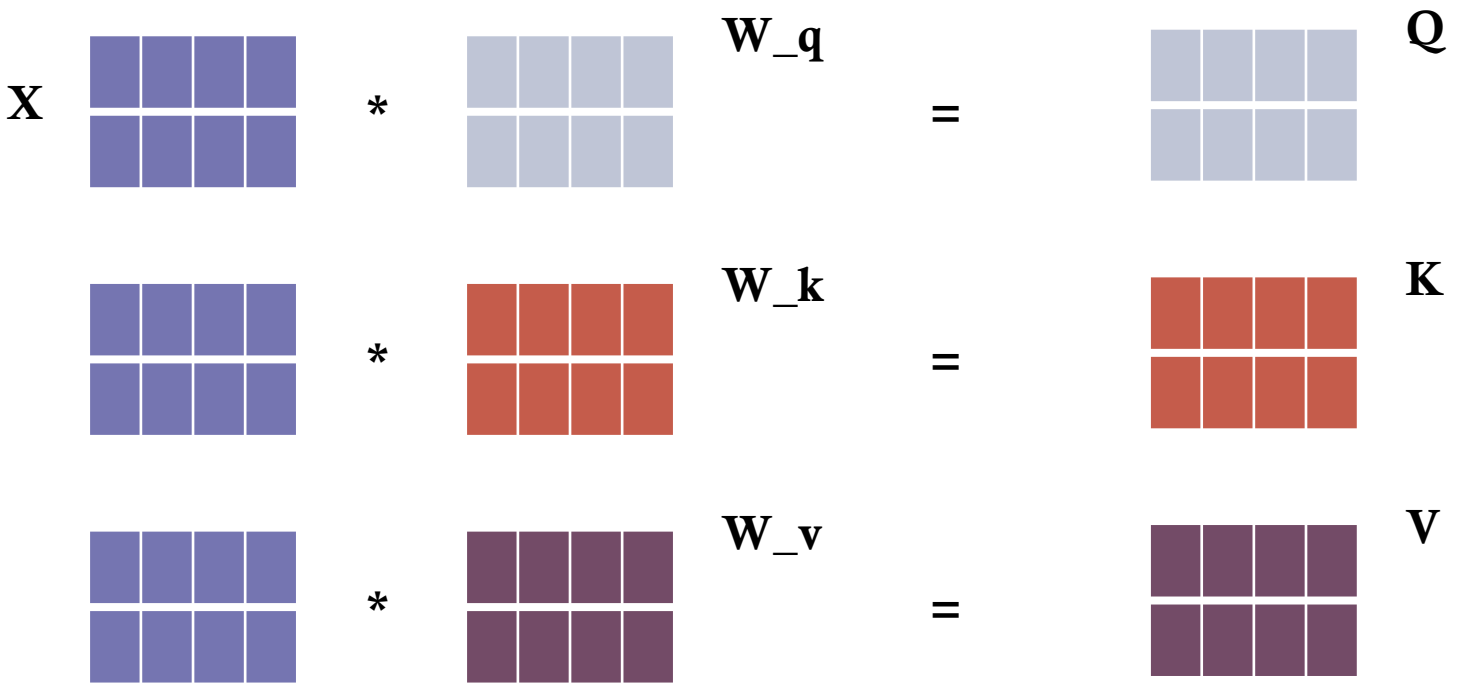
- Where:
  - **Q:** query, encodes what the current token is looking for.
  - **K:** key, encodes what the token represents to others.
  - **V:** value, encodes the actual semantic information.

MULTI-HEAD SELF ATTENTION

- The dot product between Q and K measures the **cosine similarity** between them.
- This measure is normalized and a **softmax operation** is applied, producing a probability distribution where weights sum up to 1.
- Finally, these weights are multiplied for corresponding values: this will keep high values for words strongly related to the current one. Other words will be cancelled out.

MULTI-HEAD SELF ATTENTION

- Typically, Q, K, and V are created multiplying the words embedding with **learnable matrices.**



ENCODER

- The **Encoder** is the physical block where the multi-head **self-attention mechanism** takes place. It is used to create a **context-aware representation** for each token.
- The Encoder is usually composed of a stack of identical encoders, to allow the model to capture **different relationships** within the elements of the sequence (semantic, grammar, …).
- Between different layers within the encoder there is usually a **Feed-Forward Layer.**
- This layer is used to enhance the model adding the capability of handling **non-linear data** (through the ReLU activation function).

DECODER

- The **Decoder** is the engine responsible to produce the final result.
- The architecture of the Decoder is similar to the Encoder's, with a couple of differences.
- **Masked Self-Attention:** through this layer, the Decoder looks at the data it has generated, to predict the next one. To prevent the model to look at future data, we mask them out.
- **Cross-Attention:** in this layer Queries come from the Decoder, and Keys and Values come from the Encoder. This allow the model to focus on specific parts of the original input sentence (Encoder's work) while deciding which token to generate next.
- As for the Encoder, the Decoder is usually a **stack of identical decoders,** to allow the model to switch between different levels of representation.

EXERCISES

- The goal is to implement a Transformer architecture.
- **Assignment:** download and complete *Vanilla_transformer.ipynb*, following the instructions provided in the comments